# Übungsblatt 2: RISC-V-Registerbefehle

**Papieraufgaben und schrittweises Prüfen mit Venus**

**Musterlösung** — Die Zustände sind Modellrechnungen. Die eigene Venus-Beobachtung wird im Labor protokolliert.

## Ziel und Arbeitsweise

Sie verfolgen Registerzustände, prüfen unmittelbare Konstanten und übersetzen zwischen Assemblertext und Befehlswort. Dafür genügen die Inhalte aus Vorlesung 1 und 2; C, Speicherbefehle und Funktionsaufrufe werden nicht vorausgesetzt.

Bearbeiten Sie Aufgaben 1–6 zunächst auf Papier, ohne Simulator. In Aufgabe 7 vergleichen Sie Ihre Vorhersage mit der Kursdistribution von Venus. Öffnen Sie die Musterlösung erst nach Ihrer eigenen Bearbeitung. Die **60 Punkte** der Papieraufgaben dienen der Selbstkontrolle und sind keine Prüfungsbewertung.

| Aufgabe | Schwerpunkt | Minuten | Punkte |
|---|---|---|---|
| 1 | Registerzustände und Aliasnamen | 8 | 8 |
| 2 | Wortbreite und Bitmasken | 9 | 10 |
| 3 | Schiebeoperationen | 8 | 8 |
| 4 | Immediate-Grenzen | 7 | 8 |
| 5 | Registerformat | 13 | 14 |
| 6 | Immediate-Format und Adressen | 10 | 12 |
| 7 | Venus: Gleichung und Vergleich | 30 | — |

Für vier Schulstunden zu je 45 Minuten: 75 Minuten Vorlesung einschließlich Rückfragen, 55 Minuten Papieraufgaben, 30 Minuten Venus, 10 Minuten Quiz und 10 Minuten Auswertung. Das ist eine Planungsannahme; bei größerem Unterstützungsbedarf kann Aufgabe 6 nachbesprochen werden. Die optionale Vertiefung ist zusätzliche Reserve.

## Gemeinsame Annahmen

Wir verwenden RV32I, die RISC-V-Integerbasis mit 32-Bit-Registern, ohne komprimierte Erweiterung C. Ein nativer Befehl belegt vier Byte. Adressen zählen Byte, die Kursumgebung ist Little Endian. Registerarithmetik speichert die unteren 32 Bit; `x0` liefert null und verwirft Schreibversuche. Nicht genannte Registerinhalte sind unbekannt und dürfen nicht als null vorausgesetzt werden.

Die ABI, Application Binary Interface, ist die Softwarevereinbarung für Registerrollen. Es gilt `t0=x5`, `t1=x6`, `t2=x7`, `t3=x28`, `t4=x29`, `t5=x30`, `t6=x31`, `a0=x10`, `a1=x11`. Namen bezeichnen Architekturregister; Aliasnamen erzeugen keine weiteren Speicherstellen.

## Aufgabe 1: Dieselben Register, neue Werte

**8 Punkte · 8 Minuten · Papier**

Nur für diese Aufgabe seien `t0`, `t1`, `t2` und `x0` anfangs null. Füllen Sie die Zustände jeweils **nach** der angegebenen Instruktion aus.

```
addi t0, x0, 7
addi t1, x0, 3
add  t0, t0, t1
sub  t1, t0, t1
add  t2, t1, x0
add  x0, t0, t2
```

a) Zustände der sechs Zeilen (4 Punkte). b) Welcher Wert wird in Zeile 3 als erste Quelle gelesen und warum? (1 Punkt). c) Welchen Wert zeigt `x5` nach Zeile 4? Begründen Sie. (1 Punkt). d) Welche Rechnung führt Zeile 6 aus, und was wird gespeichert? (2 Punkte).

**Musterlösung**

| nach Zeile | `t0` | `t1` | `t2` | `x0` |
|---|---|---|---|---|
| 1 | 7 | 0 | 0 | 0 |
| 2 | 7 | 3 | 0 | 0 |
| 3 | 10 | 3 | 0 | 0 |
| 4 | 10 | 7 | 0 | 0 |
| 5 | 10 | 7 | 7 | 0 |
| 6 | 10 | 7 | 7 | 0 |

b) Zeile 3 liest den alten Wert 7 aus `t0` und 3 aus `t1`. Erst danach schreibt sie 10 nach `t0`. Quelle und Ziel dürfen identisch sein.

c) `x5` zeigt 10, weil `t0` und `x5` dieselbe Registerstelle bezeichnen. Zeile 4 schreibt nur nach `t1`.

d) Die Addition berechnet $10 + 7 = 17$. `x0` verwirft das Ergebnis und bleibt null; alle anderen genannten Werte bleiben erhalten. Daraus folgt keine Aussage über die physische Anzahl der Hardwaretakte.

## Aufgabe 2: Zahlenmuster und Bitmasken

**10 Punkte · 9 Minuten · Papier**

a) Ergänzen Sie das Ergebniswort mit acht Hexziffern. Lesen Sie dieses Wort zusätzlich als signed Zweierkomplement. Geben Sie an, ob ein Carry über Bit 31 hinaus und ob signed Überlauf entsteht (je 2 Punkte).

b) Es gilt `t0=0x000000A5` und `t1=0x0000000F`. Bestimmen Sie die drei Zielwörter (je 1 Punkt).

```
and t2, t0, t1
or  t3, t0, t1
xor t4, t0, t1
```

c) Warum ist die letzte Operation keine Aussage „die Zahlen sind verschieden, also Ergebnis 1“? (1 Punkt).

**Musterlösung**

| Quellen von `add` | Ergebniswort | signed Wert | Carry | signed Überlauf |
|---|---|---|---|---|
| `0xFFFFFFFF`, 1 | `0x00000000` | 0 | ja | nein |
| `0x7FFFFFFF`, 1 | `0x80000000` | −2147483648 | nein | ja |
| `0x80000000`, `0x80000000` | `0x00000000` | 0 | ja | ja |

Die erste Rechnung ergibt vorzeichenlos $2^{32}$, signed aber $-1 + 1 = 0$. Die zweite überschreitet die größte positive signed Zahl. Die dritte ergibt signed $-2^{32}$ und verliert im Ziel alle höheren Bits. Carry und signed Überlauf beschreiben unterschiedliche Grenzen. `add` speichert nur die unteren 32 Bit und setzt kein architektonisches Überlaufflag.

b) Binär unten acht Bit: `10100101` und `00001111`. Und ergibt `00000101`, Oder `10101111`, exklusives Oder `10101010`. Also `t2=0x00000005`, `t3=0x000000AF` und `t4=0x000000AA`; alle oberen Bit sind null.

c) `xor` verknüpft jede Stelle getrennt. Das Muster `0xAA` enthält mehrere Einsen und ist keine normalisierte Wahrheitszahl 1.

## Aufgabe 3: Weite und Füllbits unterscheiden

**8 Punkte · 8 Minuten · Papier**

Es gilt `t0=0xFFFFFFFD` ($-3$), `t1=1`, `t5=32`. Jede Zeile wird unabhängig mit diesen Anfangswerten betrachtet.

a) Bestimmen Sie das Zielwort und begründen Sie die Füllung beziehungsweise effektive Weite (je 1 Punkt).

b) Welches Ergebnis hätte `sll t4, t0, t5` für `t5=33`? (1 Punkt). c) Ist `slli t4, t0, 32` zulässig? Warum unterscheidet sich der Fall von a)? (2 Punkte). d) Warum ist das arithmetische Schieben von $-3$ um eins nicht gleich einer Division durch zwei mit Abschneiden zur null? (1 Punkt).

**Musterlösung**

| Instruktion | Zielwort | Begründung |
|---|---|---|
| `srl t2, t0, t1` | `0x7FFFFFFE` | logisch rechts, links null |
| `sra t3, t0, t1` | `0xFFFFFFFE` | arithmetisch rechts, links eins |
| `sll t4, t0, t5` | `0xFFFFFFFD` | untere fünf Bits von 32 sind null |
| `srai t6, t0, 1` | `0xFFFFFFFE` | direkte Weite eins, Vorzeichenfüllung |

b) Die unteren fünf Bits von 33 ergeben eins. Ein Links-Shift von `0xFFFFFFFD` um eins liefert `0xFFFFFFFA`; das obere Bit geht verloren, rechts kommt null hinzu.

c) Nein. Die direkt angegebene Weite ist nur von 0 bis 31 codierbar. Bei einer Registerweite ist dagegen die Auswahl der unteren fünf Bits Teil der definierten Wirkung. Man darf den unzulässigen Immediate-Operanden nicht stillschweigend zu null umdeuten.

d) Arithmetisches Rechtsschieben ergibt minus zwei und entspricht hier Abrunden nach unten. Abschneiden zur null würde minus eins ergeben.

## Aufgabe 4: Konstanten und ihre Grenzen

**8 Punkte · 7 Minuten · Papier**

a) Entscheiden Sie jeweils: gültige native Instruktion, gültige Pseudoinstruktion oder unzulässiger Operand. Begründen Sie anhand der Breite beziehungsweise des Zahlenbereichs (je 1 Punkt).

b) Stellen Sie $-4$ als zwölf Bit, als drei Hexziffern und nach Vorzeichenerweiterung als acht Hexziffern dar (1 Punkt).

c) Wie ersetzen Sie eine Subtraktion von sieben durch `addi`? Warum gelingt die Subtraktion von $-2048$ nicht mit demselben Schema in einer einzelnen `addi`-Instruktion? (1 Punkt).

**Musterlösung**

| Assemblertext | Einordnung |
|---|---|
| `addi t0, x0, 2047` | nativ, obere signed Grenze |
| `addi t0, x0, 2048` | unzulässig, über der oberen Grenze |
| `addi t0, x0, -2048` | nativ, untere signed Grenze |
| `li t0, 2048` | Pseudoinstruktion, mehrere native Befehle möglich |
| `andi t0, t1, 4095` | unzulässig, logische Immediates ebenfalls signed |
| `xori t0, t1, -1` | nativ, minus eins wird zu 32 Einsen erweitert |

b) $2^{12} - 4 = 4092$, also `1111 1111 1100` = `0xFFC`. Das gesetzte Bit 11 wird in die oberen zwanzig Stellen kopiert: `0xFFFFFFFC`. Nullerweiterung zu `0x00000FFC` wäre der andere Zahlenwert 4092.

c) `addi t0, t0, -7` zieht sieben ab. Für die Subtraktion von minus 2048 wäre plus 2048 nötig, das nicht passt. Möglich ist etwa `li t1, -2048` und `sub t0, t0, t1`.

## Aufgabe 5: R-Format in beide Richtungen

**14 Punkte · 13 Minuten · Papier**

Referenz: Von Bit 31 bis Bit 0 folgen `funct7[31:25]`, `rs2[24:20]`, `rs1[19:15]`, `funct3[14:12]`, `rd[11:7]` und `opcode[6:0]`. Registerfelder enthalten je fünf Bit. Für `add`: opcode `0110011`, funct3 `000`, funct7 `0000000`. Für `sub`: gleicher opcode und funct3, aber funct7 `0100000`.

a) Codieren Sie `sub x7, x5, x6`. Zeigen Sie alle Feldwerte, die zusammengesetzten 32 Bit und das Hexwort (6 Punkte).

b) Decodieren Sie `0x407302B3`. Zeigen Sie die sechs Felder und den Befehl sowohl mit `x`-Namen als auch mit Aliasnamen. Es gilt `t0=x5`, `t1=x6`, `t2=x7` (6 Punkte).

c) In a) wird der Inhalt von `x5` von 7 auf 100 geändert. Ändert sich die Codierung? Genügt opcode `0110011` alleine zum Erkennen von `sub`? Begründen Sie beide Antworten (2 Punkte).

**Musterlösung**

a) `rd=x7` → `00111`, `rs1=x5` → `00101`, `rs2=x6` → `00110`. Die Wortreihenfolge lautet:

```
0100000 00110 00101 000 00111 0110011
0100 0000 0110 0010 1000 0011 1011 0011
   4    0    6    2    8    3    B    3
```

Ergebnis: `0x406283B3`.

b) `0x407302B3` wird an den Feldgrenzen zerlegt:

```
0100000 00111 00110 000 00101 0110011
funct7  rs2   rs1   f3  rd    opcode
```

Also `funct7=0100000`, `rs2=7`, `rs1=6`, `funct3=000`, `rd=5`, `opcode=0110011`. Die Kombination bezeichnet `sub x5, x6, x7` beziehungsweise `sub t0, t1, t2`.

c) Der Quellinhalt verändert die spätere Rechnung, nicht das Wort: Codiert wird die Nummer fünf. Opcode allein reicht nicht; erst die passenden funct3- und funct7-Werte unterscheiden diese Subtraktion beispielsweise von `add`.

## Aufgabe 6: I-Format und Instruktionsadressen

**12 Punkte · 10 Minuten · Papier**

a) Codieren Sie `addi t2, t1, -7`. Es gilt `t2=x7` und `t1=x6`. Referenz: `imm[31:20]` mit zwölf Bit, `rs1[19:15]`, `funct3[14:12]` = `000`, `rd[11:7]`, `opcode[6:0]` = `0010011`. Zeigen Sie das Zweierkomplement, alle Felder und das Hexwort (6 Punkte).

b) Gegeben ist folgende **native** Lehrentwicklung einer Konstantenladung. Der erste Befehl liegt an Adresse $B =$ `0x1000`. Notieren Sie die Adressen aller drei Befehle, die Adresse des Labels `Marke` und den PC nach dem dritten Befehl. Wie viele Byte belegen die Befehle insgesamt? (4 Punkte).

```
main:
    lui  t0, 0x1
    addi t0, t0, -2048
Marke:
    addi t1, t0, -7       # letzter Rechenbefehl
```

c) Speichern Sie das Wort aus a) in der Kursumgebung ab Adresse `0x2000`. Geben Sie die vier Byte in aufsteigender Adressreihenfolge an (2 Punkte).

### Musterlösung

a) Minus sieben ist $2^{12} - 7 = 4089$, also `0xFF9` = `111111111001`. Die Felder ergeben:

```
111111111001 00110 000 00111 0010011
1111 1111 1001 0011 0000 0011 1001 0011
   F    F    9    3    0    3    9    3
```

Ergebnis: `0xFF930393`. Die Rückrechnung des Immediate lautet $4089 - 4096 = -7$.

b) Befehlsadressen: `0x1000`, `0x1004` und `0x1008`. `Marke` benennt `0x1008`. Nach dem dritten, normal fortlaufenden Befehl ist der PC, Program Counter beziehungsweise Programmzähler, `0x100C`. Drei native Wörter belegen zwölf Byte. Die beiden Labels und der Kommentar erzeugen keine zusätzlichen Plätze. `lui` setzt hier zunächst 4096; erst der zweite Befehl stellt 2048 her.

c) Little Endian legt das niederwertige Byte zuerst ab: `0x2000: 93`, `0x2001: 03`, `0x2002: 93`, `0x2003: FF`. Das numerische Wort bleibt `0xFF930393`.

## Aufgabe 7: Die Gleichung in Venus prüfen

**30 Minuten · Rechner und Papierprotokoll**

Implementieren Sie $Y = (A + B) - (C + D)$ mit $A = 5$, $B = 10$, $C = 3$, $D = 2$. Belegen Sie `t0`–`t3` mit den Eingaben, `t4` und `t5` mit den beiden Summen und `t6` mit dem Ergebnis. Die Eingaben bleiben erhalten.

1. Sagen Sie die drei Rechenergebnisse voraus, bevor Sie Venus starten.
2. Schreiben Sie ein vollständiges Programm mit `.text`, `.globl main` und Startlabel `main:`. Initialisieren Sie mit `li`.
3. Setzen Sie das Label `Ergebnis_pruefen:` unmittelbar vor die Vorbereitung der Ausgabe. Vergleichen Sie dort die vorhergesagten und angezeigten Registerwerte.
4. Venus: Kopieren Sie `t6` nach `a1`, setzen Sie `a0=1` und führen Sie `ecall` aus. Anschließend setzen Sie `a0=10` und beenden mit `ecall`. Der erste Dienst druckt eine signed Ganzzahl, der zweite beendet das Programm. Diese Vereinbarung ist nicht die normale Funktionsrückgabe in `a0`.
5. Prüfen Sie die drei Varianten unten nach **erneutem Assemblieren und Neustart**. Notieren Sie die erste Abweichung, falls eine auftritt.

| Eingaben $(A, B, C, D)$ | erwartete Summen, $Y$ | Beobachtung vor der Ausgabe |
|---|---|---|
| $(5, 10, 3, 2)$ | selbst vorhersagen | |
| $(0, 0, 0, 0)$ | selbst vorhersagen | |
| $(5, 1, 10, 2)$ | selbst vorhersagen | |
| $(-5, 10, 3, 2)$ | selbst vorhersagen | |

Die Schrittanzeige verfolgt native Instruktionen und misst keine Hardwaretakte. Prüfen Sie die tatsächlich erzeugten Instruktionen, insbesondere bei Pseudoinstruktionen. Das Programm soll nach der Ausgabe kontrolliert enden.

**Musterlösung**

Die vollständige Datei steht in vl2_gleichung.s. Die drei Rechenzeilen sind `add t4, t0, t1`, `add t5, t2, t3` und `sub t6, t4, t5`. Vor den Diensten gilt:

| Eingaben | `t4` | `t5` | `t6` / erwartete Druckzahl |
|---|---|---|---|
| $(5, 10, 3, 2)$ | 15 | 5 | 10 |
| $(0, 0, 0, 0)$ | 0 | 0 | 0 |
| $(5, 1, 10, 2)$ | 6 | 12 | −6 = `0xFFFFFFFA` |
| $(-5, 10, 3, 2)$ | 5 | 5 | 0 |

Die Ausgabevorbereitung kopiert durch `addi a1, t6, 0`. `li a0, 1` und `ecall` drucken; `li a0, 10` und `ecall` beenden. Die Tabelle ist eine geprüfte Modellvorhersage, kein ersetztes Beobachtungsprotokoll. Ein unveränderter Quellwert ist vor dem Dienstaufruf zugesichert; die Aufgabe setzt keine pauschale Registererhaltung beliebiger Dienste voraus.

## Optionale Vertiefung: Pseudotext und Beobachtung

### Zusätzliche Reserve · nicht in den 85 Übungsminuten enthalten

a) Schreiben Sie ein vollständiges Venus-Programm, das fünf in `t1` lädt, mit einem einzigen `slli` um drei Stellen in `t2` schiebt und das Ergebnis mit `sub` nach `t3` negiert. Prüfen Sie vor dem Abschluss die drei Register. Welche Ergebnisse erwarten Sie?

b) Öffnen Sie anschließend vl2_immediates.s. Sagen Sie `s1` bis `s5` und `s8` vor dem Label `Ergebnis_pruefen` voraus. Die Aliasregister `s0`–`s11` werden hier ausschließlich im Hauptprogramm verwendet; eine aufgerufene Funktion wird nicht modelliert.

c) Untersuchen Sie in der nativen Instruktionsansicht die Entwicklung von `li s4, 2048`. Halten Sie alle erzeugten Befehle sowie den Zwischenwert nach dem ersten davon fest. Vergleichen Sie mit der gültigen Lehrentwicklung `lui s4, 0x1` und `addi s4, s4, -2048`. Beschreiben Sie die tatsächlich beobachtete Folge, auch wenn sie anders dargestellt wird.

### Musterlösung

a) Ein vollständiger Rahmen verwendet `.text`, `.globl main` und `main:`, danach:

```
li   t1, 5
slli t2, t1, 3
sub  t3, x0, t2
Ergebnis_pruefen:
li   a0, 10
ecall
```

Vor dem Abschluss: `t1=5`, `t2=40` und `t3=-40=0xFFFFFFD8`. `slli` bedeutet logisches Linksschieben mit direkter Weite. Hier geht kein relevantes Bit verloren.

b) `s1=0x00000005`, `s2=0x00000085`, `s3=0xFFFFFF5A`, `s4=2048`, `s5=50000`, `s8=0x00000ABC`. Die Maske 4095 wird für `s8` durch `li s7, 4095` bereitgestellt und anschließend mit dem Registerbefehl `and` angewendet.

c) In der Lehrentwicklung beträgt der Zwischenwert 4096 und der Endwert 2048. Das tatsächliche Venus-Protokoll ist selbst einzutragen. Die Rechnung belegt die Gültigkeit der Folge, nicht eine bestimmte Werkzeuganzeige.